1 Wide LetRec in ATL's Shallow Embedding

1.1 Definition

Wide LetRec consists of a dependent type `wide_letrec_args`; a function `run_wide_letrec_step`, which interprets a value of type `wide_letrec_args ls` as a single-step state transformer on `hlist ls` (using the helper `wl_step` to update each slot after its dependencies), where `hlist ls` is a heterogeneous list that holds the state of the system; and a wrapper `run_wide_letrec`, which applies that step repeatedly for a given number of times to an initial state.

Specifically, `wide_letrec_args` has two constructors, `WLEmpty` and `WLStore`. `WLEmpty` is the base case, and it describes an empty system. `WLStore` adds one slot of type `x` to the existing system `rest` over the slots `deps` and stores a step function of type `(hlist deps -> x -> x)`. When the system runs, this function receives the already-updated values of the slots in `deps` together with the slot's current value of type `x`, and returns its new value of type `x`. Each slot's new value therefore depends on its own previous value and on the values of the slots in `deps`, which are updated before it in each iteration.

```
Inductive wide_letrec_args : list Set -> Type :=
  | WLEmpty : wide_letrec_args nil
  | WLStore : forall (x : Set) `{TensorElem x} (deps : list Set),
     (hlist deps -> x -> x) ->
      wide_letrec_args deps ->
      wide_letrec_args (x :: deps).
```

`run_wide_letrec_step` is the update function for the whole state over one iteration. It applies a helper function `wl_step` at each slot for per-slot update. Given a step function, a recursive update function for the dependent slots, and the current state of the system, `wl_step` applies the step function to the already-updated values and the slot's current state to produce the updated value of the slot and replace the old value with the newly computed value.

```
Definition wl_step {x : Set} {deps : list Set}
  (step : hlist deps -> x -> x)
  (rec : hlist deps -> hlist deps)
  (old : hlist (x :: deps))
  : hlist (x :: deps) :=
  let new_deps := rec (htail old) in
  let x_new := step new_deps (hhead old) in
  HCons x_new new_deps.

Fixpoint run_wide_letrec_step {ls}
  (args : wide_letrec_args ls)
  : hlist ls -> hlist ls :=
  match args with
  | WLEmpty => fun _ => HNil
  | WLStore x _ deps step rest => wl_step step (run_wide_letrec_step rest)
  end.
```

`run_wide_letrec` is then a wrapper function around `run_wide_letrec_step`. It applies `run_wide_letrec_step` `steps` number of times to the inital values, to update the whole state over `steps` number of iterations.

```
Definition run_wide_letrec {ls} (args : wide_letrec_args ls) (steps : nat)
  (init : hlist ls) : hlist ls :=
  repeat_apply steps (run_wide_letrec_step args) init.
```

1.2 Example Usage of Wide LetRec

The following example computes online variance using Wide LetRec. The computation is decomposed into three steps: `mean_step`, `ssd_step`, and `variance_step`. The `ssd_step` update uses the newly updated mean, while `variance_step` uses the newly updated SSD. This illustrates how Wide LetRec allows later slots to consume values computed by earlier dependency updates within the same iteration. In `online_variance_correct`, `online_variance_args` is run for five iterations. In each iteration, the dependency slots are recursively updated before the slots that depend on them, so the mean is updated before the SSD, and the SSD is updated before the variance. Each step may additionally use its own value from the previous iteration.

```
Definition xs : list R := [2; 4; 4; 4; 5]%R.

Definition mean_step : hlist [] -> list R -> list R :=
  fun _ old => GEN [ i < 5 ] (
    (|[i =? 0]| xs _[i]) <+>
    (|[0 <? i]| (old _[i-1] + (xs _[i] - old _[i-1]) / IZR (i + 1))))%R.

Definition ssd_step : hlist [list R] -> list R -> list R :=
  fun deps old =>
    let mn := hhead deps in
    GEN [ i < 5 ] (
      (|[i =? 0]| 0) <+>
      (|[0 <? i]| (old _[i-1] + (xs _[i] - mn _[i-1]) * (xs _[i] - mn _[i]))))%R.

Definition variance_step : hlist [list R; list R] -> list R -> list R :=
  fun deps _ =>
    let ssd := hhead deps in
    GEN [ i < 5 ] (ssd _[i] / IZR (i + 1))%R.

Definition online_variance_args : wide_letrec_args [list R; list R; list R] :=
  WIDELETREC [ mean_step , ssd_step , variance_step ].

Definition online_variance_init : hlist [list R; list R; list R] :=
  HCons (zeros 5) (HCons (zeros 5) (HCons (zeros 5) HNil)).

Example online_variance_correct :
  run_wide_letrec online_variance_args 5 online_variance_init
  = HCons [0; 1; 8/9; 3/4; 24/25]%R
      (HCons [0; 2; 8/3; 3; 24/5]%R
        (HCons [2; 3; 10/3; 7/2; 19/5]%R HNil)).
Proof. compute. repeat (f_equal; try lra). Qed.
```